A Comparison of Inflation Forecasting Models – Individual Report

Individual Report for COMP5530

SANDRA GURAN, University of Leeds, United Kingdom

CCS Concepts: • **Computing methodologies** → **Supervised learning**; **Neural networks**; • **Information systems** → **Time series analysis**.

Additional Key Words and Phrases: time series forecasting, RNN, GRU, deep learning

1 INTRODUCTION

This report outlines my individual contribution to our group project, *A Comparison of Inflation Forecasting Models* [Bignall 2025]. The goal of the project was to implement, evaluate, and compare a diverse set of forecasting models across classical statistical, machine learning, and deep learning paradigms. In the initial phase, the group conducted a presentation on LSTM-based forecasting [Hochreiter and Schmidhuber 1997] to gain foundational experience with time series models. I led the section on Recurrent Neural Networks (RNNs) [Elman 1990], which set the stage for my primary responsibility throughout the rest of the project. In this report, I focus specifically on RNN and GRU, an improved variant of RNN designed to address its limitation, both of which were key components of my implementation and analysis.

I was the primary contributor to the design, implementation, and optimisation of three forecasting models: a standard RNN, a Gated Recurrent Unit (GRU) model [Cho 2014], and an XGBoost regressor [Chen and Guestrin 2016]. GRU, a streamlined variant of RNN, was selected for its improved memory retention. I iteratively refined these models through baseline development, hyperparameter tuning, migration to the PyTorch Forecasting framework [Forecasting 2021], and experimentation with alternative libraries such as Darts [Herzen 2022], ensuring compatibility with the team's data pipeline and modular codebase.

My work included feature engineering, training configuration, Optuna-based hyperparameter optimisation [Devs 2019], and support for multi-horizon evaluation [Box 2015]. I also authored the method sections for these models in the group report. The full implementation is available in the shared GitHub repository [Bignall et al. 2025], where I was solely responsible for the `Training/RNN`, `GRU`, and `XGBoost` directories. This report focuses on RNN and GRU as representative sequence-based forecasting models.

2 BACKGROUND RESEARCH

My work focused on implementing and evaluating Recurrent Neural Network (RNN) architectures for inflation forecasting, specifically a standard RNN and a Gated Recurrent Unit (GRU), a variant of RNN designed to improve training stability and sequence retention.

Traditional forecasting approaches, such as ARIMA and SARIMAX, have been widely used for modelling inflation and macroeconomic indicators [Box 2015; Korstanje 2021]. However, these models often struggle to capture the non-linear and long-range dependencies present in economic time series [Bontempi 2013].

Author's address: Sandra Guran, sc21samg@leeds.ac.uk, University of Leeds, Woodhouse Lane, Leeds, West Yorkshire, United Kingdom, LS2 9JT.

To address these limitations, I investigated recurrent neural networks designed for sequence learning. RNNs [Elman 1990] maintain temporal context through their internal state but are prone to vanishing gradients, which can hinder learning over longer sequences. As a result, I also implemented the GRU [Cho 2014], a variant of RNN that introduces gating mechanisms to better regulate memory flow. GRUs are less computationally demanding than LSTMs [Hochreiter and Schmidhuber 1997], while offering improved stability during training, making them well-suited for mid-range forecasting tasks.

For model development, I used PyTorch Forecasting, a high-level library built on PyTorch Lightning that simplified training and evaluation pipelines. While I briefly explored the Darts library, I ultimately preferred PyTorch Forecasting for its flexibility and better integration with our team's infrastructure. Hyperparameter tuning was performed using Optuna, allowing me to efficiently optimise learning rates, hidden sizes, and network depth across different forecasting horizons.

This research and tooling informed my model selection, grounding the RNN and GRU implementations in both theory and practical forecasting requirements.

3 METHOD / DESCRIPTION

My contribution focused on implementing and optimising two deep learning models for inflation forecasting: a Recurrent Neural Network (RNN) and a Gated Recurrent Unit (GRU). GRU was developed as an extension of the RNN baseline to improve performance and training stability, as it incorporates gating mechanisms to better capture long-term dependencies.

Data Preparation and Feature Engineering. The monthly inflation series was enhanced with lagged values (`y_lag_1`, `y_lag_3`, `y_lag_6`), rolling statistics (mean and standard deviation over 3, 6, and 12 months), and seasonal encodings (sine/cosine of month index). All features were scaled using `StandardScaler`.

Model Implementation. The RNN was first developed in PyTorch with batch-first input sequences. It served as a baseline to evaluate sequential modelling performance. I later implemented the GRU model, which introduced update and reset gates to mitigate vanishing gradient issues. Both models were restructured within the PyTorch Forecasting framework to enable modular training, exogenous feature support, and direct multi-horizon output.

Model parameters included:

- `input_size`: number of input features,
- `hidden_size`: tuned between 32 and 512,
- `num_layers`: 1 to 6,
- `output_size`: corresponding to the forecast horizon (1, 3, 6, or 12 months).

Training used the Adam optimiser (learning rate in [1e-5, 1e-1]), MSE loss, batch size 32, and input sequence length of 12. An 80/20 train/validation split was used, with multi-step forecasting targets aligned accordingly.

Motivation for GRU. The GRU model was introduced as an improvement over the vanilla RNN. Its gating architecture allows it to retain relevant information over longer sequences while being computationally lighter than LSTMs, making it well-suited for forecasting tasks involving medium-term dependencies.

Hyperparameter Tuning. I used Optuna to optimise model architecture and training parameters, running 200 trials per model per horizon. Early pruning was applied during 10-epoch trials, and the best configurations were retrained over 50 epochs. Training logs and model checkpoints were saved per horizon.

Output Integration. All forecasts were saved in `.npy` format and incorporated into the team's central metrics and visualisation framework for evaluation alongside other models.

4 RESULTS

The RNN and GRU models I implemented were evaluated across rolling forecast horizons of 1, 3, 6, and 12 months. As shown in Fig. 1, RNN slightly outperformed GRU at Horizon 1 (RMSE = 4.49 vs. 7.33), while GRU was better at Horizon 3 (RMSE = 7.34 vs. 7.86), likely due to its gating mechanism for handling short-term dependencies. However, both recurrent models exhibited instability, reacting sensitively to recent fluctuations, indicating limited robustness in dynamic economic settings.

Beyond three months, performance declined notably. At Horizon 6, GRU reached RMSE = 4.27 (R^2 = −32.9) and RNN 8.52 (R^2 = −133.9), revealing the compounding effect of forecast error and vanishing gradients. By Horizon 12, both models degraded further, with GRU and RNN exceeding RMSE of 12 and R^2 values dropping below −296 (Fig. 3).

Optuna tuning yielded consistent improvements of 5–10% across horizons (Fig. 2), particularly for RNN and GRU, where learning rate and depth significantly impacted training stability. The use of exogenous features provided modest gains at shorter horizons by enhancing responsiveness to macroeconomic variation.

Overall, the GRU model performed better than RNN across most horizons, particularly at medium and long range, due to its ability to retain information more effectively and reduce vanishing gradient issues. While both models were carefully optimised, they were ultimately outperformed by architectures such as NHITS, which delivered more stable and accurate results across all horizons. Nonetheless, RNN and GRU provided valuable benchmarks that helped contextualise the performance of more advanced models under consistent evaluation conditions.

5 SELF REFLECTION

This project deepened my understanding of time series forecasting and practical deep learning through hands-on model development and integration. Taking a lead role on the RNN components, a key class of models in neural forecasting, gave me the opportunity to shape a core part of the comparison and contribute meaningfully to the project's direction. It also strengthened my ability to write modular code, apply tools like Optuna and PyTorch Forecasting, and collaborate effectively. Working in a team improved my communication and boosted my confidence in navigating technical challenges and leading on shared objectives.

REFERENCES

G. et al. Bignall. 2025. A Comparison of Inflation Forecasting Models. Univ. of Leeds.

Bignall et al. 2025. COMP5530M Group Project – Inflation Forecasting. https://github.com/sc20geb/COMP5530M-Group-Project-Inflation-Forecasting.

G. et al. Bontempi. 2013. ML Strategies for Time Series Forecasting. In *Eur. Bus. Intell. Summer School.* 62–77.

G. et al. Box. 2015. *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.

T. Chen and C. Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proc. ACM SIGKDD.* 785–794.

K. et al. Cho. 2014. RNN Encoder–Decoder for Statistical Machine Translation. In *Proc. EMNLP.* 1724–1734.

Optuna Devs. 2019. Optuna: Hyperparameter Optimization.

J. L. Elman. 1990. Finding Structure in Time. *Cogn. Sci.* 14, 2 (1990), 179–211.

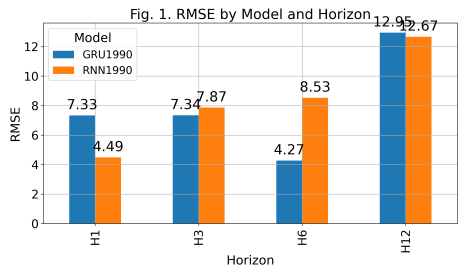
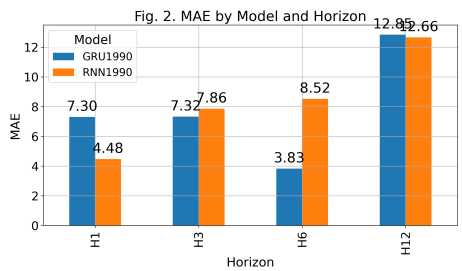
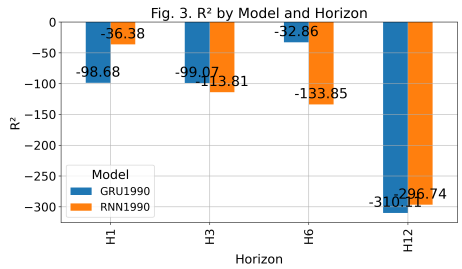
PyTorch Forecasting. 2021. Time Series Forecasting with PyTorch.

J. et al. Herzen. 2022. Darts: Modern ML for Time Series. *J. Mach. Learn. Res.* 23, 124 (2022), 1–6.

S. Hochreiter and J. Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (1997), 1735–1780.

J. Korstanje. 2021. *Practical Time Series Analysis.* Addison-Wesley.

6 HOW ETHICAL ISSUES ARE ADDRESSED

No ethical issues arose during this project. All data was publicly available and anonymised, and all models were developed using open-source libraries with proper citation.

A ADDITIONAL EVALUATION FIGURES